

2.2.1

下面我将详细解析这段代码的每一部分，包括函数功能、参数含义和执行流程：

1. 导入库

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
import pickle
from sklearn.metrics import classification_report
from imblearn.over_sampling import SMOTE
```

- `pandas`: 数据处理库
- `train_test_split`: 数据集划分工具
- `LogisticRegression`: 逻辑回归模型
- `pickle`: Python对象序列化模块
- `classification_report`: 分类评估报告生成器
- `SMOTE`: 过采样方法处理不平衡数据

2. 数据加载与预览

```
data = pd.read_csv('finance数据集.csv')
print(data.head())
```

- `pd.read_csv()`: 读取CSV文件
 - 参数: 文件路径
- `head()`: 显示前5行数据
- 功能: 加载数据并初步检查

3. 特征与目标分离

```
x = data.drop(['SeriousDlqin2yrs', 'Unnamed: 0'], axis=1)
y = data['SeriousDlqin2yrs']
```

- `drop()`: 移除指定列
 - `axis=1`: 按列操作
 - 移除目标列和无关索引列
- 功能: 创建特征矩阵X和目标向量y

4. 数据集划分

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split()`: 划分训练/测试集
 - `test_size=0.2`: 测试集占比20%
 - `random_state=42`: 随机种子（保证可重复性）

5. 初始模型训练

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

- `LogisticRegression()`: 创建逻辑回归模型
 - `max_iter=1000`: 最大迭代次数
- `fit()`: 模型训练方法
 - 参数: 训练集特征和目标

6. 模型保存

```
with open('2.2.1_model.pkl', 'wb') as file:
    pickle.dump(model, file)
```

- `pickle.dump()`: 序列化保存模型
 - `'wb'`: 以二进制写入模式打开文件
 - 保存到'2.2.1_model.pkl'

7. 初始预测与评估

```
y_pred = model.predict(X_test)
pd.DataFrame(y_pred, columns=['预测结果']).to_csv('2.2.1_results.txt', index=False)

report = classification_report(y_test, y_pred, zero_division=1)
with open('2.2.1_report.txt', 'w') as file:
    file.write(report)

accuracy = (y_test == y_pred).mean()
print(f"模型准确率: {accuracy:.2f}")
```

- `predict()`: 生成预测结果
- `classification_report()`: 生成分类报告
 - `zero_division=1`: 处理除零警告
- 手动计算准确率
- 保存预测结果和评估报告

8. 处理数据不平衡

```
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

- `SMOTE()`: 创建过采样器
 - `random_state=42`: 随机种子
- `fit_resample()`: 生成平衡数据
 - 通过合成少数类样本来平衡数据

9. 重新训练与评估

```
model.fit(X_resampled, y_resampled)
y_pred_resampled = model.predict(X_test)

pd.DataFrame(y_pred_resampled, columns=['预测结果']).to_csv('2.2.1_results_xg.txt',
index=False)

report_resampled = classification_report(y_test, y_pred_resampled, zero_division=1)
with open('2.2.1_report_xg.txt', 'w') as file:
    file.write(report_resampled)

accuracy_resampled = (y_test == y_pred_resampled).mean()
print(f"重新采样后的模型准确率: {accuracy_resampled:.2f}")
```

- 使用平衡数据重新训练模型
- 生成新的预测和评估报告
- 比较处理前后的准确率

关键点说明:

1. **数据不平衡处理**: 原始金融数据中违约样本 (`SeriousDlqin2yrs=1`) 通常远少于正常样本, SMOTE通过合成少数类样本来改善这个问题。
2. **模型评估**: 使用`classification_report`输出精确率、召回率、F1值等指标, 比单纯准确率更能反映不平衡数据的模型表现。
3. **文件输出**:
 - `.pkl`: 保存训练好的模型对象
 - `.txt`: 保存预测结果和评估报告
4. **随机种子**: 多处设置`random_state=42`保证实验可重复性

这段代码完整展示了从数据准备到模型训练、评估、优化的全流程, 特别关注了金融领域常见的数据不平衡问题及其解决方案。

2.2.2

下面我将详细解析这段代码的每一部分，包括函数功能、参数含义和执行流程：

1. 导入库

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import pickle
from sklearn.ensemble import RandomForestRegressor
```

- `pandas`：数据处理库
- `train_test_split`：数据集划分工具
- `LinearRegression`：线性回归模型
- `StandardScaler`：数据标准化处理器
- `Pipeline`：机器学习流水线工具
- `pickle`：Python对象序列化模块
- `RandomForestRegressor`：随机森林回归模型

2. 数据加载与预览

```
df = pd.read_csv('auto-mpg.csv')
print(df.head())
```

- `pd.read_csv()`：读取CSV文件
 - 参数：文件路径('auto-mpg.csv')
- `head()`：显示前5行数据
- 功能：加载数据并初步检查数据结构

3. 数据清洗

```
df['horsepower'] = pd.to_numeric(df['horsepower'], errors='coerce')
df = df.dropna()
```

- `pd.to_numeric()`：将列转换为数值类型
 - `errors='coerce'`：将无效值转为NaN
- `dropna()`：删除包含NaN的行
- 功能：处理horsepower列中的非数值和缺失值

4. 特征工程

```
x = df[['cylinders', 'displacement', 'horsepower', 'weight', 'acceleration', 'model year',  
'origin']]  
y = df['mpg']
```

- 选择7个特征作为自变量(X)
- 选择mpg(每加仑英里数)作为因变量(y)
- 功能：构建特征矩阵和目标向量

5. 数据集划分

```
x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split()`：划分训练/测试集
 - `test_size=0.2`：测试集占20%
 - `random_state=42`：固定随机种子保证可重复性

6. 构建线性回归流水线

```
pipeline = Pipeline([('scaler', StandardScaler()), ('linreg', LinearRegression())])
```

- `Pipeline()`：创建处理流水线
 - 第一步：`StandardScaler` 标准化（均值为0，方差为1）
 - 第二步：`LinearRegression` 线性回归
- 功能：自动完成数据标准化和模型训练

7. 模型训练与保存

```
pipeline.fit(x_train, y_train)  
with open('2.2.2_model.pkl', 'wb') as model_file:  
    pickle.dump(pipeline, model_file)
```

- `fit()`：训练流水线（自动先标准化再回归）
- `pickle.dump()`：序列化保存整个流水线
 - 包含标准化器和训练好的回归模型

8. 预测与评估

```
y_pred = pipeline.predict(X_test)
results_df = pd.DataFrame(y_pred, columns=['预测结果'])
results_df.to_csv('2.2.2_results.txt', index=False)

with open('2.2.2_report.txt', 'w') as results_file:
    results_file.write(f'训练集得分: {pipeline.score(X_train, y_train)}\n')
    results_file.write(f'测试集得分: {pipeline.score(X_test, y_test)}\n')
```

- `predict()`：生成预测结果
- `score()`：计算 R^2 决定系数
 - 训练集得分反映拟合程度
 - 测试集得分反映泛化能力
- 保存预测结果和评估分数

9. 随机森林模型

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)
results_rf_df = pd.DataFrame(y_pred_rf, columns=['预测结果'])
results_rf_df.to_csv('2.2.2_results_rf.txt', index=False)

with open('2.2.2_report_rf.txt', 'w') as results_rf_file:
    results_rf_file.write(f'训练集得分: {rf_model.score(X_train, y_train)}\n')
    results_rf_file.write(f'测试集得分: {rf_model.score(X_test, y_test)}\n')
```

- `RandomForestRegressor()`：创建随机森林模型
 - `n_estimators=100`：100棵决策树
 - `random_state=42`：固定随机种子
- 相同流程进行训练、预测和评估
- 功能：与线性回归模型对比性能

关键点说明：

1. **数据标准化**：在流水线中自动完成，避免数据泄露
2. **模型对比**：线性回归（简单模型）vs 随机森林（复杂模型）
3. **评估指标**：使用 R^2 分数（越接近1表示模型越好）
4. **可复现性**：全程设置`random_state=42`
5. **模型持久化**：保存完整流水线（包含预处理步骤）

这段代码完整展示了从数据预处理到模型训练、评估、比较的全流程，特别展示了：

- 使用Pipeline简化机器学习流程

- 两种不同复杂度模型的实现与对比
- 规范的模型保存和结果记录方法

2.2.3

下面我将详细解析这段代码的每一部分，包括函数功能、参数含义和执行流程：

1. 导入库

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
import pickle
from sklearn.metrics import mean_squared_error, r2_score
import xgboost as xgb
```

- `pandas`：数据处理库
- `train_test_split`：数据集划分工具
- `RandomForestRegressor`：随机森林回归模型
- `pickle`：Python对象序列化模块
- `mean_squared_error` 和 `r2_score`：回归评估指标
- `xgboost`：XGBoost梯度提升库

2. 数据加载与预览

```
df = pd.read_csv('fitness_analysis.csv')
print(df.head())
```

- `pd.read_csv()`：读取CSV文件
 - 参数：文件路径('fitness_analysis.csv')
- `head()`：显示前5行数据
- 功能：加载数据并初步检查数据结构

3. 数据清洗

```
df = df.applymap(lambda x: x.strip() if isinstance(x, str) else x)
df.columns = df.columns.str.strip()
```

- `applymap()`：对整个DataFrame应用函数
 - `lambda` 函数：去除字符串字段前后空格
- `str.strip()`：去除列名中的空格
- 功能：统一数据格式，避免因空格导致的处理问题

4. 特征工程

```
x = df[['Your gender', 'How important is exercise to you ?', 'How healthy do you consider yourself?']]
x = pd.get_dummies(x) # 将分类变量转为数值变量
y = df['Your age'].apply(lambda x: int(x.split(' ')[0]))
```

- `pd.get_dummies()`：将分类变量转为哑变量(one-hot编码)
- `apply()`：处理年龄数据
 - `split(' ')[0]`：假设年龄格式为"25 years"，提取数字部分
 - `int()`：转换为整数
- 功能：构建数值型特征矩阵和目标向量

5. 数据集划分

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split()`：划分训练/测试集
 - `test_size=0.2`：测试集占20%
 - `random_state=42`：固定随机种子保证可重复性

6. 随机森林模型

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
```

- `RandomForestRegressor()`：创建随机森林模型
 - `n_estimators=100`：100棵决策树
 - `random_state=42`：固定随机种子
- `fit()`：训练模型

7. 模型保存

```
with open('2.2.3_model.pkl', 'wb') as model_file:
    pickle.dump(rf_model, model_file)
```

- `pickle.dump()`：序列化保存模型
 - `'wb'`：二进制写入模式
 - 保存到'2.2.3_model.pkl'

8. 预测与评估

```
y_pred = rf_model.predict(X_test)
results_df = pd.DataFrame(y_pred, columns=['预测结果'])
results_df.to_csv('2.2.3_results.txt', index=False)

train_score = rf_model.score(X_train, y_train)
test_score = rf_model.score(X_test, y_test)
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
with open('2.2.3_report.txt', 'w') as report_file:
    report_file.write(f'训练集得分: {train_score}\n')
    report_file.write(f'测试集得分: {test_score}\n')
    report_file.write(f'均方误差(MSE): {mse}\n')
    report_file.write(f'决定系数(R^2): {r2}\n')
```

- `predict()`: 生成预测结果
- `score()`: 计算 R^2 决定系数
- `mean_squared_error()`: 计算均方误差
- `r2_score()`: 计算 R^2 分数
- 保存预测结果和多种评估指标

9. XGBoost模型对比

```
xgb_model = xgb.XGBRegressor(n_estimators=100, random_state=42)
xgb_model.fit(X_train, y_train)
y_pred_xgb = xgb_model.predict(X_test)

results_df_xgb = pd.DataFrame(y_pred_xgb, columns=['预测结果'])
results_df_xgb.to_csv('2.2.3_results_xgb.txt', index=False)

with open('2.2.3_report_xgb.txt', 'w') as xgb_report_file:
    xgb_report_file.write(f'XGBoost训练集得分: {xgb_model.score(X_train, y_train)}\n')
    xgb_report_file.write(f'XGBoost测试集得分: {xgb_model.score(X_test, y_test)}\n')
    xgb_report_file.write(f'XGBoost均方误差(MSE): {mean_squared_error(y_test, y_pred_xgb)}\n')
    xgb_report_file.write(f'XGBoost决定系数(R^2): {r2_score(y_test, y_pred_xgb)}\n')
```

- `XGBRegressor()`: 创建XGBoost模型
 - 参数与随机森林类似
- 相同流程进行训练和评估
- 功能: 与随机森林模型进行性能对比

关键点说明：

1. 数据预处理：

- 字符串清理确保数据一致性
- 分类变量转换为数值特征（one-hot编码）
- 年龄数据特殊处理

2. 模型对比：

- 随机森林（传统集成方法）
- XGBoost（梯度提升方法）
- 使用相同评估标准进行公平比较

3. 评估指标：

- R²分数（解释方差比例）
- MSE（误差平方的期望值）
- 同时记录训练集和测试集表现

4. 工程实践：

- 固定随机种子保证可复现性
- 规范保存模型和结果
- 完整的模型训练-评估-比较流程

2.2.4

下面我将详细解析这段代码的每一部分，包括函数功能、参数含义和执行流程：

1. 导入库

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import joblib
from xgboost import XGBRegressor
```

- `pandas`：数据处理库，用于数据读取和操作
- `train_test_split`：数据集划分工具，用于拆分训练集和测试集
- `LinearRegression`：线性回归模型
- `mean_squared_error` 和 `r2_score`：回归模型评估指标
- `joblib`：Python对象序列化模块（替代pickle，更适合大数据）
- `XGBRegressor`：XGBoost回归模型

2. 数据加载与预览

```
data = pd.read_excel('大学生低碳生活行为的影响因素数据集.xlsx')
print(data.head())
```

- `pd.read_excel()`：读取Excel文件
 - 参数：文件路径
- `head()`：显示前5行数据
- 功能：初步检查数据结构和内容

3. 数据清洗与特征工程

```
data_cleaned = data.drop(columns=['序号', '所用时间'])
data_cleaned = pd.get_dummies(data_cleaned, drop_first=True)
```

- `drop()`：删除指定列
 - 移除无关的序号和所用时间列
- `get_dummies()`：将分类变量转换为哑变量
 - `drop_first=True`：避免哑变量陷阱（减少一列）

4. 定义特征和目标

```
target = '5.您进行过绿色低碳的相关生活方式吗?'
X = data_cleaned.drop(columns=[target])
y = data_cleaned[target]
```

- 指定目标变量（低碳行为）
- 创建特征矩阵X（删除目标列）
- 创建目标向量y

5. 数据集划分

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

- `train_test_split()`：划分数据集
 - `test_size=0.2`：测试集占20%
 - `random_state=42`：固定随机种子

6. 线性回归模型

```
model = LinearRegression()
model.fit(X_train, y_train)
```

- `LinearRegression()`：创建普通最小二乘线性回归模型

- `fit()`：训练模型

7. 模型保存

```
model_filename = '2.2.4_model.pkl'  
joblib.dump(model, model_filename)
```

- `joblib.dump()`：序列化保存模型
 - 比pickle更高效，特别适合包含大数组的模型

8. 预测与评估（线性回归）

```
y_pred = model.predict(X_test)  
results = pd.DataFrame({'实际值': y_test, '预测值': y_pred})  
results.to_csv('2.2.4_results.txt', index=False, sep='\t')  
  
with open('2.2.4_report.txt', 'w') as f:  
    f.write(f'均方误差: {mean_squared_error(y_test, y_pred)}\n')  
    f.write(f'决定系数: {r2_score(y_test, y_pred)}\n')
```

- `predict()`：生成预测结果
- `mean_squared_error()`：计算MSE（越小越好）
- `r2_score()`：计算R²（越接近1越好）
- 保存预测结果和评估指标

9. XGBoost模型（对比改进）

```
xgb_model = XGBRegressor(  
    n_estimators=1000,  
    learning_rate=0.05,  
    max_depth=5,  
    subsample=0.8,  
    colsample_bytree=0.8  
)  
xgb_model.fit(X_train, y_train)
```

- `XGBRegressor()`：创建XGBoost回归模型
 - `n_estimators=1000`：1000棵决策树
 - `learning_rate=0.05`：收缩步长
 - `max_depth=5`：树的最大深度
 - `subsample=0.8`：样本采样比例
 - `colsample_bytree=0.8`：特征采样比例

10. XGBoost预测与评估

```
y_pred_xg = xgb_model.predict(X_test)
results_xg = pd.DataFrame({'实际值': y_test, '预测值': y_pred_xg})
results_xg.to_csv('2.2.4_results_xg.txt', index=False, sep='\t')

with open('2.2.4_report_xgb.txt', 'w') as f:
    f.write(f'均方误差: {mean_squared_error(y_test, y_pred_xg)}\n')
    f.write(f'决定系数: {r2_score(y_test, y_pred_xg)}\n')
```

- 相同评估流程，与线性回归结果对比

关键点说明：

1. 特征工程：

- 分类变量使用one-hot编码
- 删除无关特征减少噪声

2. 模型对比：

- 线性回归（基线模型）
- XGBoost（高级集成方法）
- 使用相同评估标准（MSE和R²）

3. XGBoost参数：

- 学习率控制每棵树的贡献
- 树深度防止过拟合
- 行列采样增加多样性

4. 工程实践：

- 使用joblib保存模型（适合scikit-learn）
- 结果文件使用制表符分隔（避免逗号冲突）
- 完整记录模型性能指标

这段代码展示了从数据预处理到模型训练、评估、改进的完整流程，特别适用于社会科学领域的回归分析问题。通过两种模型的对比，可以分析不同算法对低碳行为预测的效果差异。

2.2.5

```
import pandas as pd
```

导入 Pandas 库并简称为 `pd`。Pandas 是 Python 中用于数据处理和分析的核心库，提供 DataFrame 等数据结构。

```
from sklearn.model_selection import train_test_split
```

从 Scikit-learn 的 `model_selection` 模块导入 `train_test_split` 函数。该函数用于将数据集随机划分为训练集和测试集。

```
from sklearn.tree import DecisionTreeRegressor
```

从 Scikit-learn 的 `tree` 模块导入决策树回归模型类 `DecisionTreeRegressor`。用于建立回归模型（预测连续值）。

```
import pickle
```

导入 Python 内置的 `pickle` 模块。用于将 Python 对象（如训练好的模型）序列化保存到文件，实现模型持久化。

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

从 Scikit-learn 的 `metrics` 模块导入三个回归评估指标：

- `mean_squared_error`：均方误差（MSE）
- `mean_absolute_error`：平均绝对误差（MAE）
- `r2_score`：决定系数（R²）

```
df = pd.read_csv('fitness_analysis.csv')
```

使用 Pandas 的 `read_csv` 函数读取名为 "fitness_analysis.csv" 的 CSV 文件，并将其加载到 DataFrame 对象 `df` 中。

```
print(df.head())
```

显示 DataFrame 的前 5 行数据。用于快速检查数据结构和内容（列名、数据类型、示例值等）。

```
x = df[['Your gender ', 'How important is exercise to you ?', 'How healthy do you consider yourself?']]
```

从 DataFrame 中选择三个特征列作为自变量（特征矩阵）：

1. 性别（Your gender）
2. 运动重要性（How important is exercise to you?）
3. 自评健康程度（How healthy do you consider yourself?）

```
x = pd.get_dummies(x)
```

使用独热编码（One-Hot Encoding）将分类变量（如性别）转换为数值形式：

- 例如：性别列 ["Male", "Female"] 会被转换为两列 "gender_Male" 和 "gender_Female"，用 0/1 表示
- 避免类别变量被误认为有序数值

```
y = df['daily_steps']
```

将 'daily_steps'（每日步数）列设置为目标变量（因变量）。这是要预测的连续值。

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

将数据集划分为训练集和测试集：

- `test_size=0.2`：20% 数据作为测试集
- `random_state=42`：固定随机种子确保结果可复现
- 返回四个对象：训练特征、测试特征、训练标签、测试标签

```
dt_model = DecisionTreeRegressor(random_state=42)
```

初始化决策树回归模型：

- `random_state=42`：控制决策树中的随机选择，确保结果可复现

```
dt_model.fit(X_train, y_train)
```

在训练集上训练决策树模型：

- 学习特征（X_train）与目标值（y_train）之间的关系
- 构建树结构（递归分割数据）

```
with open('2.2.5_model.pkl', 'wb') as model_file:  
    pickle.dump(dt_model, model_file)
```

将训练好的模型保存到文件：

- `'wb'`：以二进制写入模式打开文件
- `pickle.dump()`：将模型对象序列化到文件
- 保存为 "2.2.5_model.pkl"，可在后续直接加载使用

```
y_pred = dt_model.predict(X_test)
```

使用训练好的模型对测试集进行预测：

- 输入测试特征（X_test）
- 输出预测的每日步数（y_pred）

```
results = pd.DataFrame({'实际值': y_test, '预测值': y_pred})
```

创建包含两列的 DataFrame：

- "实际值"：测试集的真实目标值（y_test）
- "预测值"：模型的预测结果（y_pred）

```
results_filename = '2.2.5_results.txt'  
results.to_csv(results_filename, index=False, sep='\t')
```

将预测结果保存为制表符分隔的文本文件：

- `index=False`：不保存行索引
- `sep='\t'`：使用制表符（tab）作为分隔符（非逗号）

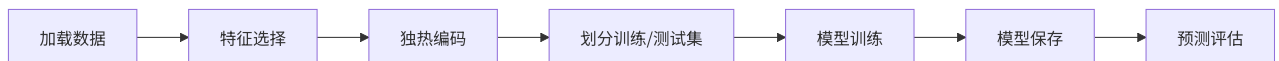
```
report_filename = '2.2.5_report.txt'
with open(report_filename, 'w') as f:
    f.write(f'均方误差: {mean_squared_error(y_test, y_pred)}\n')
    f.write(f'平均绝对误差: {mean_absolute_error(y_test, y_pred)}\n')
    f.write(f'决定系数: {r2_score(y_test, y_pred)}\n')
```

创建评估报告：

1. 计算三个指标：
 - **均方误差 (MSE)**：预测值与实际值差的平方均值（越小越好）
 - **平均绝对误差 (MAE)**：预测误差绝对值的均值（单位与目标值相同）
 - **决定系数 (R^2)**：模型解释的数据方差比例（0-1，越大越好）
2. 将指标写入文本文件 "2.2.5_report.txt"

补充说明

1. 数据处理流程



2. 模型选择原因

决策树适合处理：

- 数值和类别混合特征
- 非线性关系
- 直观的可解释性（可通过 `tree.plot_tree` 可视化）

3. 潜在改进方向

- 特征工程（处理缺失值/特征缩放）
- 超参数调优（`max_depth`, `min_samples_split`）
- 尝试其他模型（随机森林、梯度提升树）
- 交叉验证评估稳定性